Paris sportif- Beat the Bookmakers

Alexandre ROUET – Charles BOUDRY – David QUERIN – Marielle Odile GAHUNGU

Table des matières

Introduction au projet 3
Contexte 3
Rapport d'exploration et de visualisation des données 4
I - Exploration des données 4
Compréhension des données 4
Données – Valeurs manquantes 5
Stratégies 6
Conclusion 18
II – Pré-traitement des données et modèle 19
Etape 1 – Nettoyer les données 21
Etape 2 – discrétisation de la variable cible 21
Etape 3 - Sélection des variables (feature selection) 21
Etape 4 – Preprocessing 21
Etape 5 – Identification du type de problème (discrimination, clustering…) et choisir un algorithme 22
III - Modèle 23
1 - Entrainer les modèles au choix 23
2 - Evaluation de la performance des modèles entrainés et calcul des gains 28
3 - Evaluer les performances de l'algorithme (Validité des structurés trouvées) 33

Introduction au projet

Contexte

Ce projet a pour objectif de proposer une rentabilité positive lors de paris sportif sur le tennis. Le jeu de données concerne l'intégralité des matchs recensés par l'ATP depuis 2001 est fourni, ainsi que les cotes initiales proposées par plusieurs bookmakers. Notre travail consistera à analyser les données et à proposer des stratégies à rendement fiable.

Ce projet se déroulera donc en 2 temps, à savoir :

- Une étape d'exploration, de visualisation.
- Un pré-traitement des données et un modèle prédictif suggérant les cotes à jouer à une date t de telle manière que le retour sur investissement soit positif.

Rapport d'exploration et de visualisation des données

L'objectif de cette partie est de comprendre les données de la base d'étude et d'en évaluer la qualité. Pour cela nous nous sommes basés sur les techniques univariées, bivariées voire multivariées afin de garantir la qualité et l'adéquation du dataset qui servira à la modélisation de notre problématique.

En effet, la présence d'anomalies ou de valeurs manquantes peuvent influer sur le choix du modèle. Par exemple, la présence de valeurs extrêmes rend impossible l'application des méthodes d'analyse discriminante de Fischer ou de régression linéaire. Par ailleurs, si l'objet d'études est la réalisation des cartes de Kohonen, on pourra se passer du prétraitement des valeurs manquantes car elles les supportent parfaitement.

I- Exploration des données

Compréhension des données

Pour ce projet, nous disposons de 2 bases de données (ATP, Confidence).

La base ATP comporte 57 420 matchs décrits selon 22 variables d'études :

- ✓ 6 portent sur les tournois dont 1 variable quantitative et 5 variables catégorielles (ATP, Location, Tournament, Séries, Court, Surface)
- ✓ 9 portent sur les joueurs dont 3 variables quantitatives et 6 variables catégorielles (Round, Best of, Winner, Loser, WRank, LRank, Wsets, Lsets, Comment)
- ✓ 4 portent sur les bookmakers, toutes quantitatives : PSW, PWL, B365W, B365L
- ✓ 3 sur l'elo, score arbitraire attribué à chaque joueur : elo_winner, elo_loser, proba_elo

On observe que les données sont extraites par script sur un site public qui est toujours actif et à jour.

Les données sont issues directement des résultats recensés par l'ATP, ce qui les rendent fiable, à l'exception des cotes des bookmakers et de l'elo.

L'elo n'est pas une information officielle mais une donnée calculée selon la base du nombre de parties gagnées et perdues.

La base Confidence, contient des données calculées suite à l'exécution d'un modèle prédictif sur la base ATP. Il s'agit d'une approche proposée par le détenteur du notebook sur Kaggle, et nous choisissons de l'écarter de notre analyse dans un premier temps pour nous concentrer sur les données initiales.

Données – Valeurs manquantes

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 57420 entries, 2001-01-01 to 2023-08-06
Data columns (total 22 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   ATP         57420 non-null  int64
 1   Location    57420 non-null  object
 2   Tournament  57420 non-null  object
 3   Series      57420 non-null  object
 4   Court       57420 non-null  object
 5   Surface     57420 non-null  object
 6   Round       57420 non-null  object
 7   Best of     57405 non-null  float64
 8   Winner      57420 non-null  object
 9   Loser       57420 non-null  object
 10  WRank       57420 non-null  int64
 11  LRank       57420 non-null  int64
 12  Wsets       57118 non-null  float64
 13  Lsets       57116 non-null  float64
 14  Comment     57420 non-null  object
 15  PSW         48234 non-null  float64
 16  PSL         48234 non-null  float64
 17  B365W       53359 non-null  float64
 18  B365L       53382 non-null  float64
 19  elo_winner  57420 non-null  float64
 20  elo_loser   57420 non-null  float64
 21  proba_elo   57420 non-null  float64
dtypes: float64(10), int64(3), object(9)
memory usage: 10.1+ MB
```

Nous constatons la présence de valeurs manquantes.

Les valeurs manquantes sont réparties sur 7 variables dans les proportions suivantes :

```
PSL           16.00
PSW           16.00
B365W          7.07
B365L          7.03
Wsets          0.53
Lsets          0.53
Best of        0.03
```

Les données manquantes sont donc principalement des matchs sans cotes. On observe également que les données manquantes sont dans les années 2001 à 2004, c'est-à-dire sur nos données les plus anciennes.

Pour l'heure, il serait prématuré de les supprimer, un match sans cote contient des informations de valeurs quant à la performance d'un joueur. La stratégie et le modèle choisi ultérieurement dicteront les opérations de prétraitement que nous effectuerons.

A noter également que notre base ne comporte pas de données dupliquées.

Stratégies

Notre donnée cible dépend de la stratégie utilisée pour répondre à l'énoncé. Plusieurs hypothèses sont posées :

1. Mettre en concurrence les cotes de plusieurs bookmakers sur un même pari de manière à être positif quel que soit le résultat
2. Pour un bookmaker donné, mettre en place une stratégie pour ne parier que sur les matchs avec un écart de côtes très élevés
3. Evaluer les performances des joueurs pour déterminer la probabilité de victoire par joueur, et le corréler aux cotes des bookmakers pour assurer un retour sur investissement

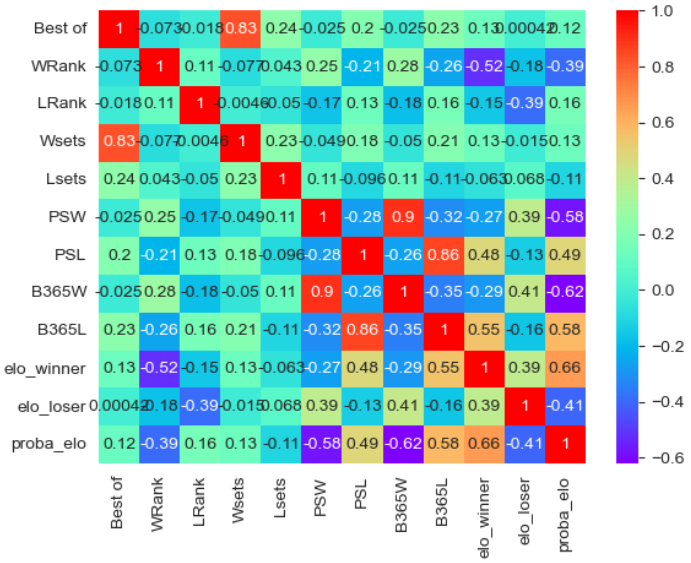
Stratégie 1 : mise en concurrence des cotes entre bookmakers

Le but de cette approche est de déterminer si, pour un même pari, la cote des bookmakers s'oppose de telle manière que parier simultanément sur les deux garantirait un bénéfice. On note que pour ce faire la cote des 2 paris doit être supérieure à 2 : En 22 ans, il n'existe que 22 matchs sur lesquels les bookmakers divergent avec une cote supérieure à 2. Le pari simultané chez les deux bookmakers garantit un gain à 100%.

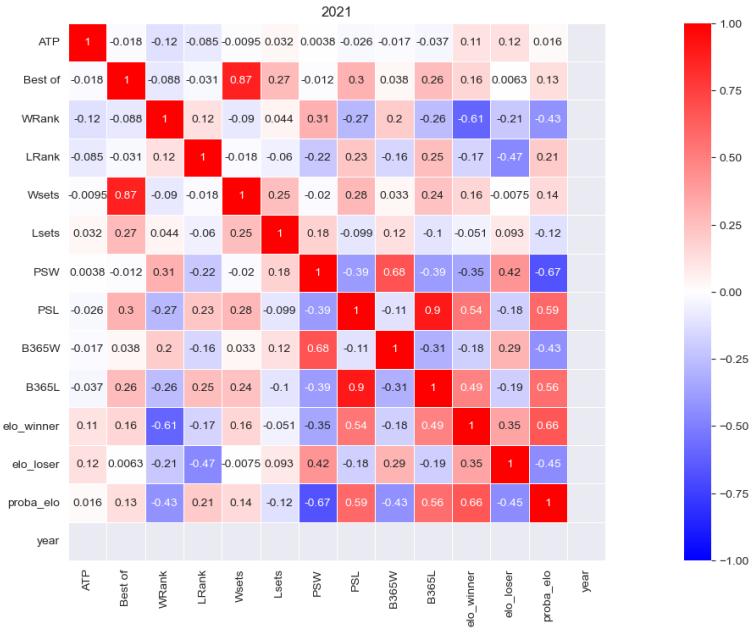
On note cependant que les bookmakers prennent parfois la liberté d'annuler des paris pour « faute de cote ». Notre stratégie devient alors très risquée car elle implique de miser gros et seul le pari perdant risque de ne pas être annulé.

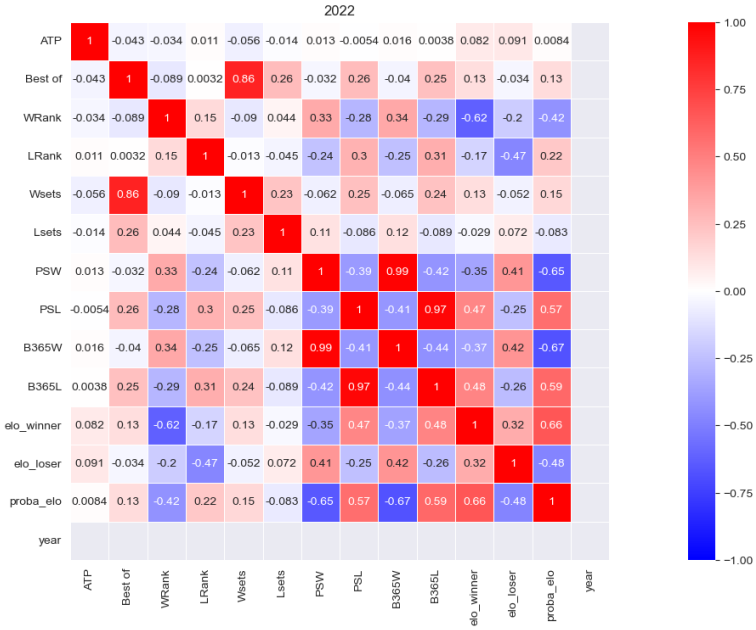
La faible fréquence de pari nous incite également à trouver une autre stratégie.

Outre ces 22 cas rares, nous observons une forte corrélation entre les cotes des bookmakers en notre possession B365W/PSW vs B365L/PSL (R^2>90%) : on en déduit que les cotes des différents bookmakers sont très souvent similaires voire identiques.

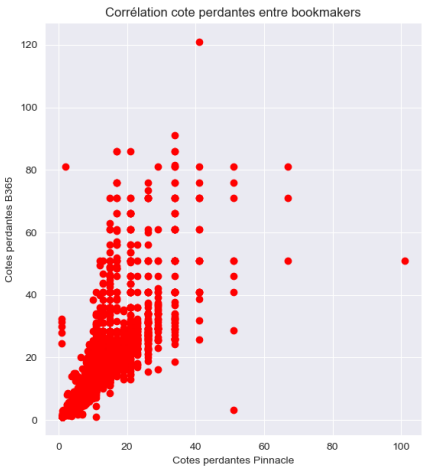


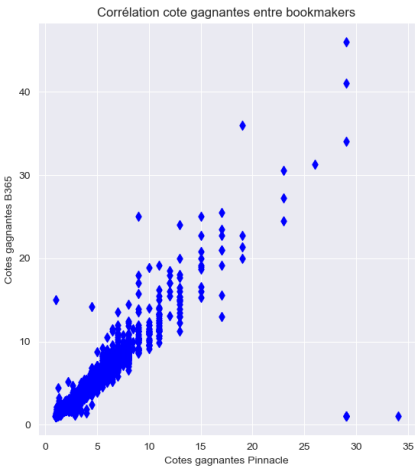
Une analyse temporelle, confirme cette tendance dans le temps :





De même, pour un même bookmaker, on observe moins de disparités pour les cotes des joueurs gagnants comparés à celles des perdants. En effet, les cotes des joueurs gagnants sont plus corrélées que les cotes perdantes. Le choix du bookmaker pourrait donc avoir son importance pour limiter les pertes des paris perdus.





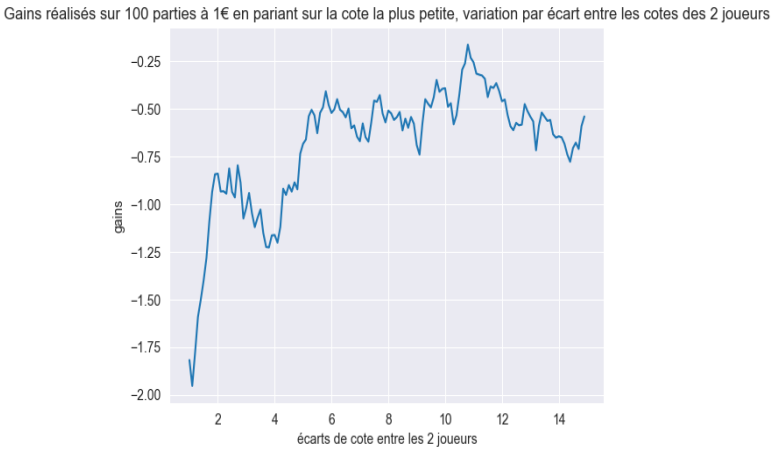
Nos conclusions sur l'analyse des données au bénéfice de l'hypothèse 1 sont donc les suivantes :

- La stratégie 1 n'est pas viable dans le temps
- Les bookmakers ont des cotes similaires, et le choix de l'un plutôt que l'autre influencera peu un modèle prédictif
- Il reste néanmoins bénéficiable de comparer les cotes des différents bookmakers pour identifier celui qui en moyenne fournit les cotes les plus intéressantes pour le joueur. On note cependant que les données des bookmakers ne sont pas Français et que les bookmakers français ne publient pas leur cote historique.

Stratégie 2 : Ne parier que sur les matchs avec un écart de cotes élevés pour un bookmaker donné

Le but de cette stratégie serait de déterminer si, pour bookmaker donné, le fait de parier uniquement sur les matchs avec un écart de cotes élevés, permettrait de prédire au mieux la probabilité d'un joueur gagnant un match.

Avec cette stratégie, nous avons constaté que, sur les gains réalisés sur 100 parties en fonction de l'écart de cote entre deux joueurs, le parieur n'a que des pertes. On en conclut que cette stratégie n'est pas viable afin d'obtenir un gain convenable surtout en pariant toujours sur la cote la plus petite.



Stratégie 3 : Approche basée sur la performance des joueurs pour déterminer le résultat des matchs

Cette stratégie est plus complexe et se divise en deux parties :

- Prédire en utilisant les données propres aux joueurs (classement mondial, nombre de sets gagnés, nombre de points avant le tournois...) et les conditions du tournois (type de surface, de séries) le vainqueur du match.
- Vérifier que la cote associée à chaque pari réussi permet un retour sur investissement

En tenant en compte le fait que le tennis se dispute entre deux joueurs ayant des conditions de jeu identique, notre analyse se porte sur les axes suivants :

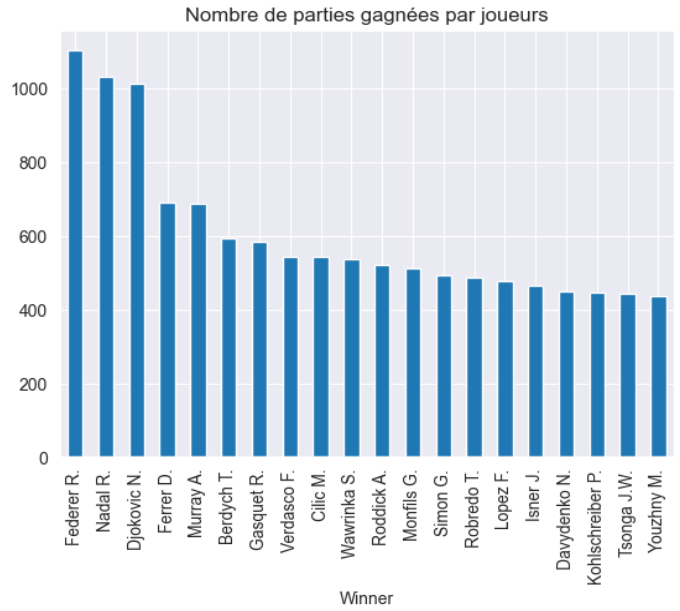
- Ratio de parties gagnées au nombre de matchs
- Influence du court (extérieur/intérieur)
- Influence de la surface (clay / hard / carpet or grass)
- Influence de la série (grand slam, masters, international)
- Influence de la localisation
- Différence de rang ATP entre les joueurs

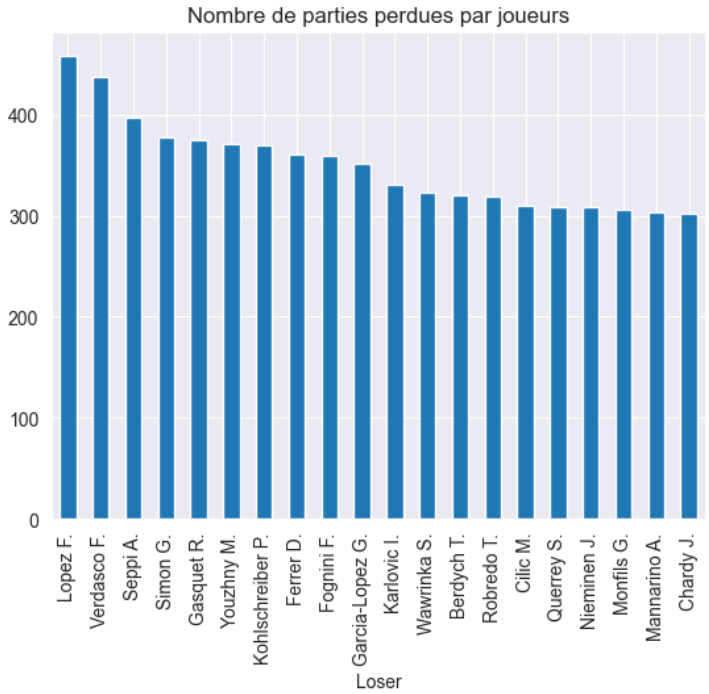
On note aussi que cette méthodologie permet d'apporter un grand nombre d'informations supplémentaires qui ne sont pour l'heure pas présent dans le jeu de données :

- Le fait que le joueur soit droitier ou gaucher
- Influence de l'âge du joueur
- Grand écart de victoire ou non (2 set gagnés 0 perdu ?)

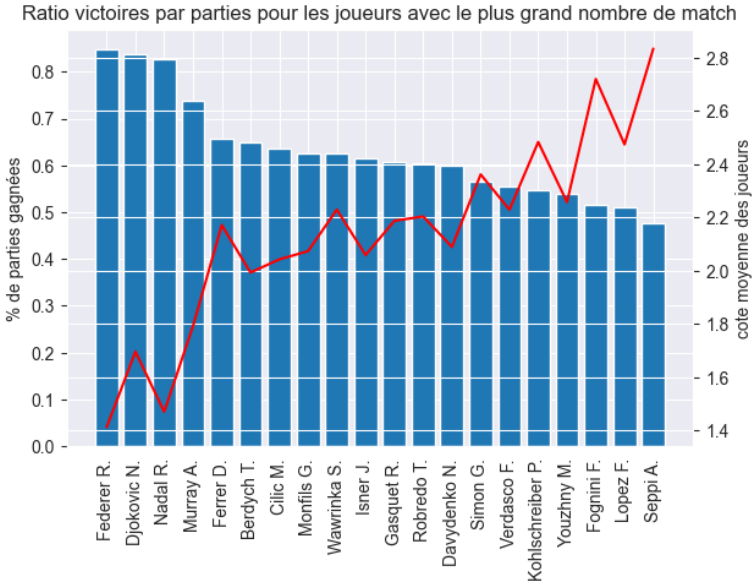
Influence des ratios de parties gagnées au nombre de matchs

Avant cela analysons les caractéristiques de notre jeu de données pouvant influer sur la performance de nos joueurs.

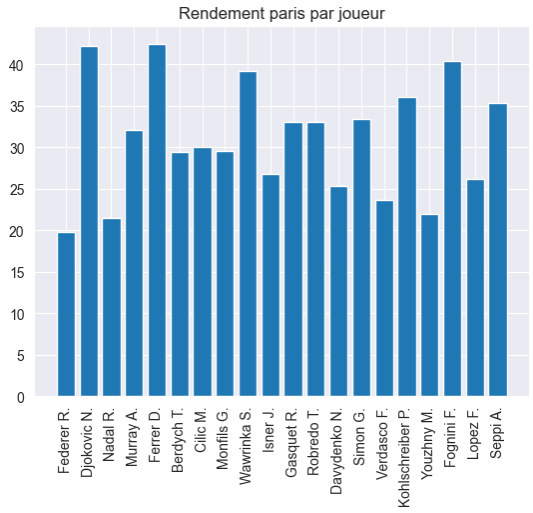


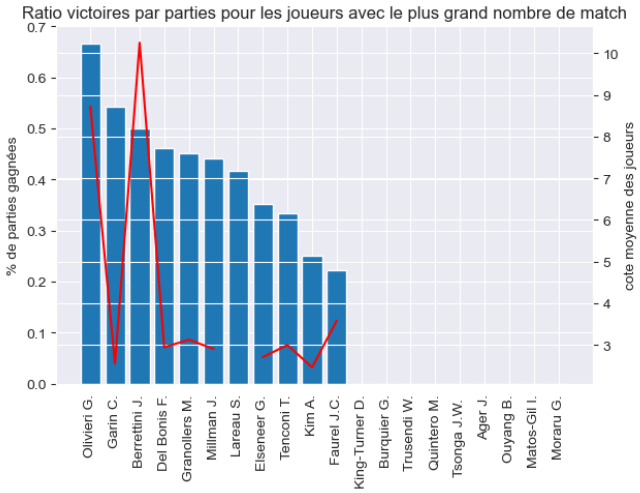


Sur ces graphiques, nous pouvons voir les 20 meilleurs joueurs de notre jeu de données et ceux ayant le plus de défaites. Nous serons donc plus attentifs envers ces joueurs en raison de leur nombre de victoires. Les autres éléments du jeu de donnée nous donneront plus d'informations si le match a lieu entre les meilleurs joueurs.

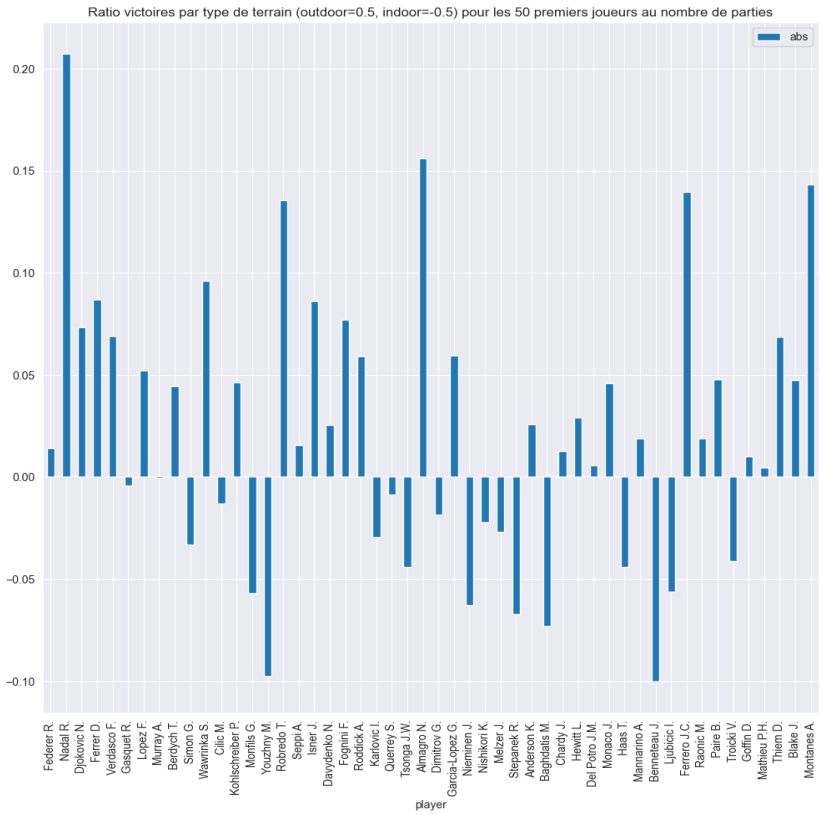


Sur ce graphique montrant le ratio de victoire nous pouvons voir que les 10 meilleurs joueurs détiennent un ratio supérieur à 0,6. Autrement dit leur constance au cours du temps est assez bonne. Nous remarquons aussi que plus le niveau de joueur diminue, plus la cote moyenne du joueur augmente. Cela signifie que la plupart des bookmakeurs suppose d'une victoire des meilleurs joueurs. En cas de victoire d'un joueur ayant un niveau moyen ou bas sur un meilleur joueur sera donc récompensé par un meilleur gain pour le risque pris grâce à cette cote élevée.





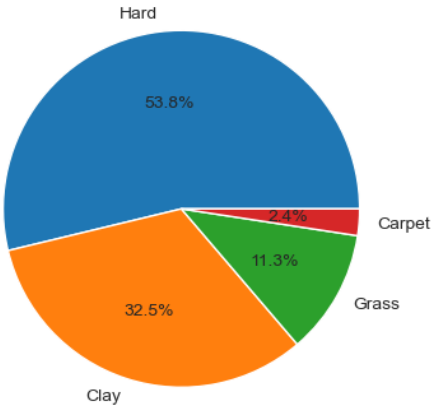
Le rendement des paris sportifs n'est pas lié au niveau des joueurs. En effet, nous pouvons voir que Novak Djokovic et Seppi A détiennent par exemple un meilleur rendement que Federer alors que nous avons un nombre de victoire de la part de Federer supérieur à Seppi.



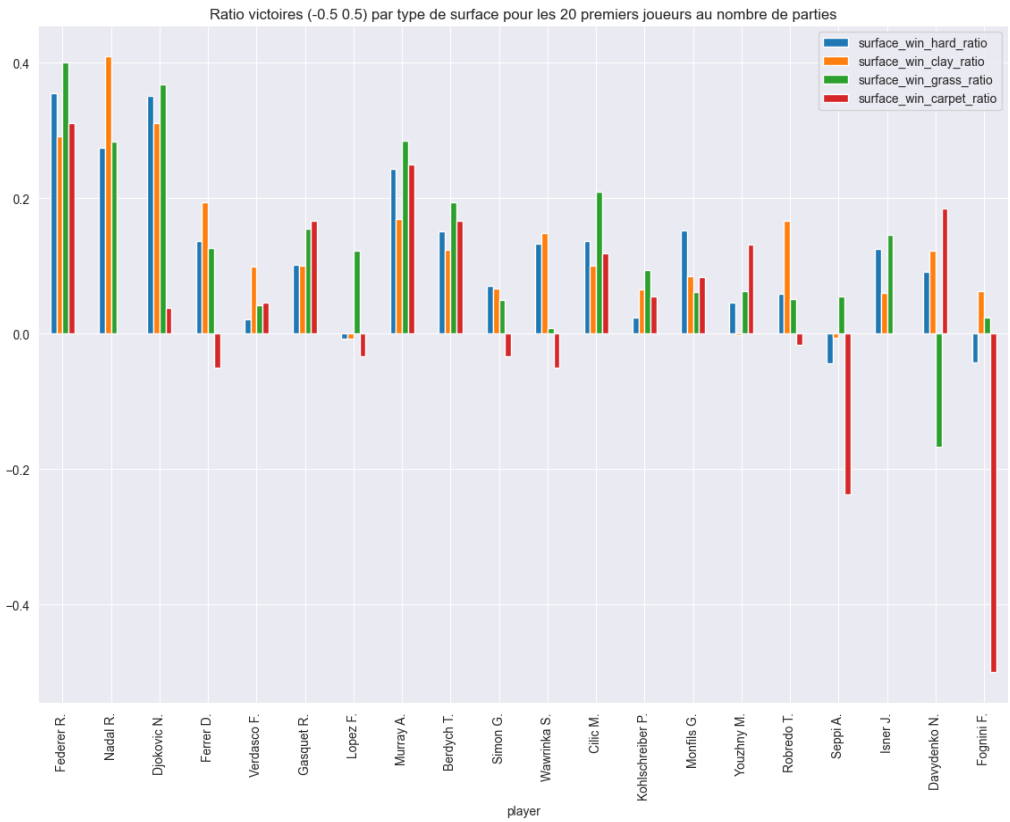
Sur ce graphique le nombre de victoires par type de terrain fluctue en fonction du joueur. Nous remarquons que Nadal détient un score assez élevé de victoires sur un court à l'extérieur contrairement à Youzhny et Benneteau qui préfèrent les courts intérieurs. Cette information peut être utile lors de la confrontation de de certains joueurs afin de savoir qui pourrait gagner grâce à ces préférences.

Influence de la surface sur les joueurs

Sur le graphique ci-dessous nous pouvons nous rendre compte que la majorité des terrains de tennis sont des sols durs ou en argile. Nous pouvons voir que les terrains à moquette sont minoritaires et ne sont même plus utilisé depuis 2009.

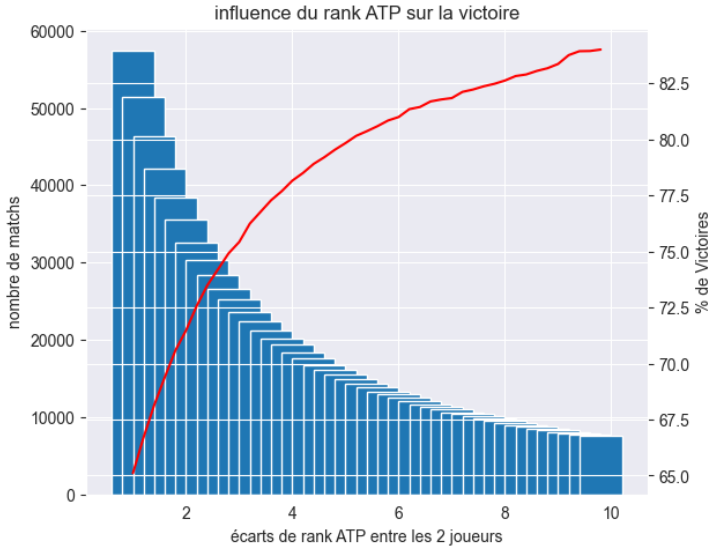


Si nous prenons en compte la surface de terrain sur le ratio de victoires des 20 meilleurs joueurs, nous pouvons voir que le type de terrain a une réelle influence sur les performances des joueurs. Les meilleurs joueurs ont de meilleures performances sur un terrain en argile comme Nadal et d'autres sur gazon comme Federer. Nous pouvons voir que les joueurs comme Federer Djokovic Nadal et Murray sont assez polyvalents et possèdent une réussite assez homogène quel que soit le type de terrain.



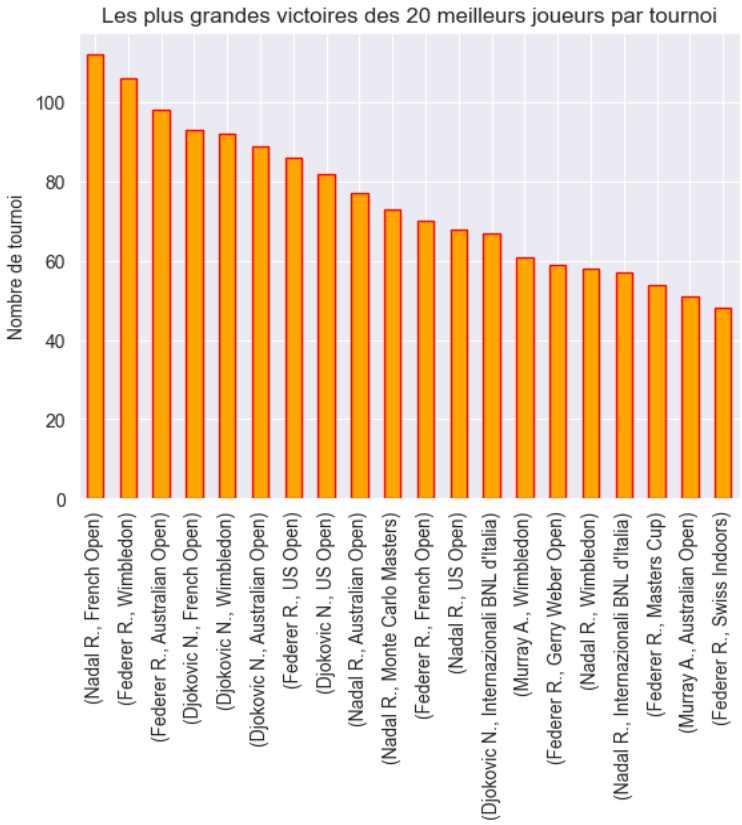
Influence du Rang ATP

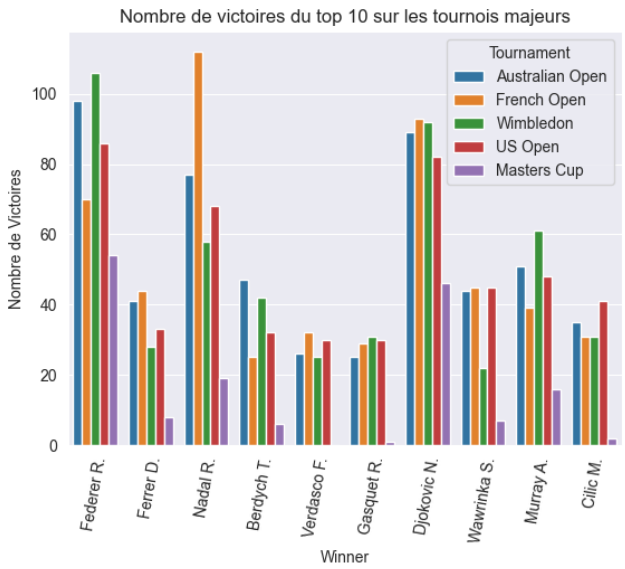
L'écart de rang ATP entre les joueurs augmente la probabilité de victoire. En effet plus cette différence de rang est criante et plus nous voyons une probabilité de gain élevée. Le rang ATP est donc un facteur important à ne pas négliger dans notre analyse de victoire d'un joueur.

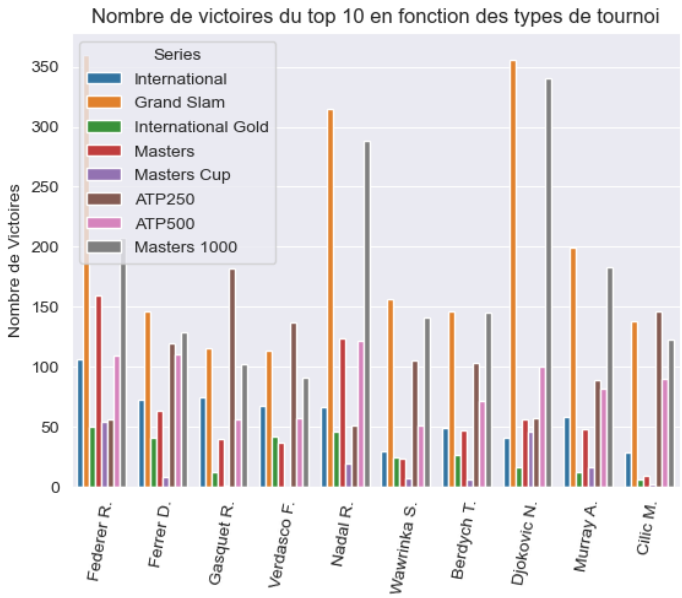


Analyse des victoires par tournoi

L'analyse confirme que le type de tournoi influe sur la prédiction du gagnant







Sur ces trois graphiques nous pouvons voir les préférences des meilleurs joueurs en fonction des tournois par exemple Nadal a de nettes facilités sur le tournoi français là ou Federer brille beaucoup plus à l'Open d'Australie et Wimbledon. De plus ces graphiques confirment les tendances découvertes lors de l'analyse de des ratios de victoires par types de terrain. Nous pouvons clairement voir la polyvalence de Djokovic qui détient des résultats assez homogènes en fonction du type de tournoi majeurs qu'il effectue. Cette variable nous sera donc utile afin de déterminer un gagnant lors d'une future confrontation. Nous notons que le nombre de victoires pour les tournois internationaux est plus faible pour la plupart des joueurs surement en raison de la rareté des tournois ou en raison de la pression lors de ces compétitions.

Conclusion

La stratégie 3 : « Approche basée sur la performance des joueurs pour déterminer le résultat des matchs » est la plus sérieuse et la plus prometteuse.

L'analyse des données associées permettent de confirmer l'influence de plusieurs feature à notre disposition sur le résultat, ce qui est encourageant.

La suite de notre rapport présentera les modèles étudiés et leurs traitements des données associées pour optimiser au mieux cette stratégie.

II – Pré-traitement des données et modèle

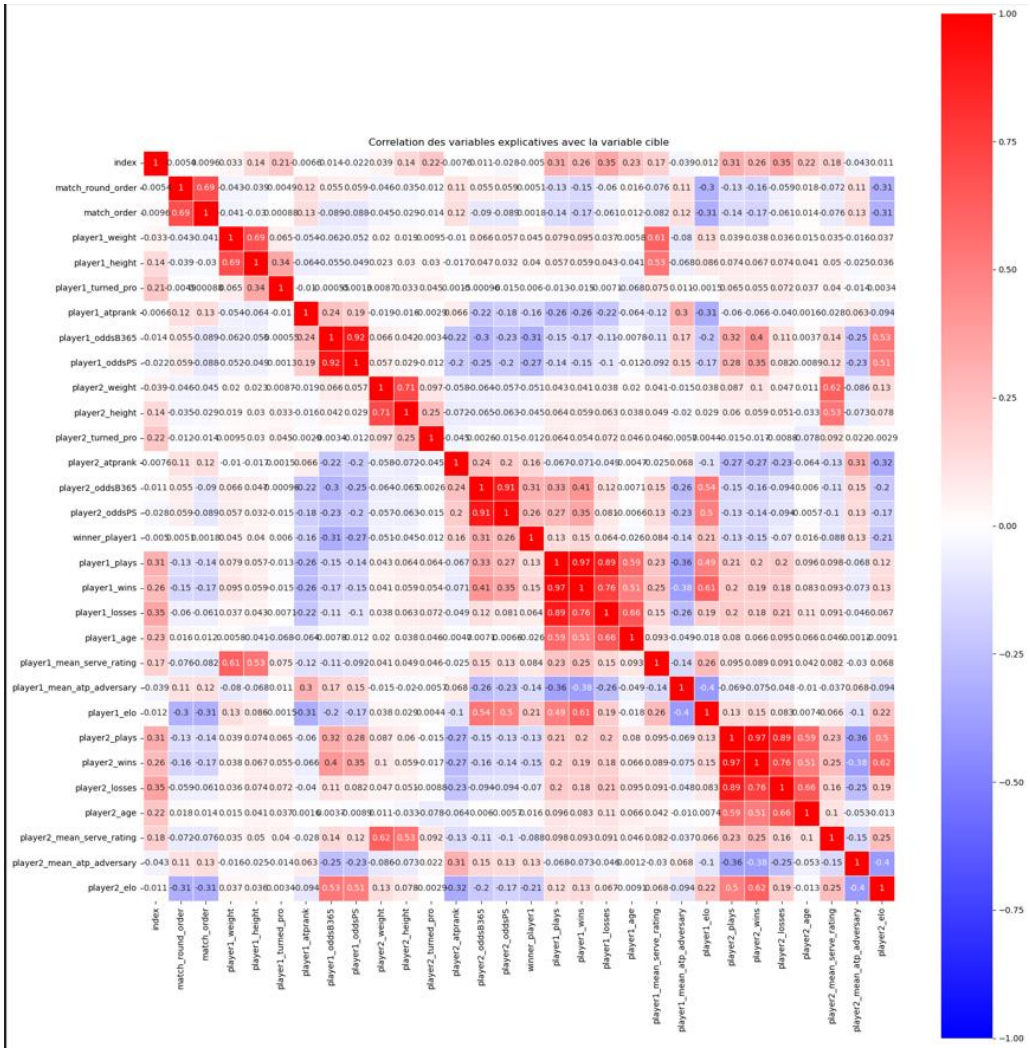
Le prétraitement des données est une étape indispensable, il nous faut :

- Créer une variable cible : “winner_player1” : variable binaire, le joueur 1 a-t-il remporté le match ? Oui = 1, Non = 0
- Agréger les informations des 2 joueurs qui réalisent le match
- Corriger ou supprimer les informations inexactes ou aberrantes afin d’améliorer la cohérence, fiabilité et valeur des données.

Les données qui constituent notre training dataset sont :

- match_date : date du match
- match_round_order
- Player1_name Player2_name : Nom et prénom des joueur 1 et joueur 2
- player1_age, player2_age : Age des joueur 1 et joueur 2
- player1_atprank, player2_atprank : Rang ATP des joueur 1 et joueur 2
- player1_plays, player2_plays : Nombre de partie jouées pour les joueurs 1 et joueur 2
- player1_wins, player2_wins : Nombre parties gagnées par les joueurs 1 et joueur 2
- player1_losses, player2_losses: Nombre parties perdues par les joueurs
- player1_elo, player2_elo: Elo par joueur
- player1_mean_serve_rating, player2_mean_serve_rating:
- player1_height, player2_height: taille des joueur 1 et joueur 2
- player1_weight, player2_weight: poids des joueur 1 et joueur 2
- player1_oddsB365, player2_oddsB365: cotes du Bookmaker B365
- player1_mean_atp_adversary, player2_mean_atp_adversary

Corrélation des features avec la variable cible



Les cotes des Bookmakers (PS et B365) sont fortement corrélées. Par conséquent, nous utiliserons uniquement les côtes d'un seul bookmaker dans notre modèle, B365 est choisi arbitrairement.

Les variables suivantes sont les plus corrélées avec la cible :

- playerX_plays: 0.97
- playerX_losses: 0.76
- playerX_elo: 0.61
- playerX_age: 0.51

Au vu de la matrice de corrélation, les cotes sont faiblement corrélées avec la variable cible.

Nous confronterons l'analyse ci-dessous aux résultats de la modélisation

Nettoyer les données

Proportion de valeurs manquantes par variable :

player1_oddsPS	16.67
player2_oddsPS	16.67
player1_oddsB365	7.62
player2_oddsB365	7.62
player1_elo	6.29
player2_elo	6.27
player1_mean_serve_rating	3.78
player2_mean_serve_rating	3.78
player1_mean_atp_adversary	3.49
player2_mean_atp_adversary	3.48
player1_turned_pro	1.04
player2_turned_pro	1.01
player1_height	0.35
player1_weight	0.31
player2_height	0.31
player2_weight	0.30
player1_birthdate	0.03
player1_age	0.03
player2_birthdate	0.02
player2_age	0.02

Les cotes ayant un rôle très important dans le calcul du ROI nous ne pouvons pas nous permettre d'approximer les valeurs manquantes. De plus, il est observé que les cotes manquantes proviennent des années les plus anciennes dans le jeu de donnée, il est donc décidé de supprimer ces matchs.

Discrétisation de la variable cible

La variable cible étant une variable binaire, sa discrétisation n'est pas nécessaire

Sélection des variables (*feature selection*)

Nous avons essayé d'autres stratégies pour améliorer la performance du modèle retenu :

- Intégration de la temporalité avec la moyenne mobile en optimisant la taille de la fenêtre mobile (3mois, 12 mois)
- Entrainement du modèle sur les données réduites avec les algorithmes PCA, t-SNE
- Ajout de nouvelles features :
    - Variables sur le tournoi :
        - Type du tournoi (player1_tourney_type, player2_tourney_type)
        - Type de surface des terrains (player1_tourney_surface, player2_tourney_surface)
    - Nombre de doubles fautes (player1_double_faults, player2_double_faults)
    - Nombre total de points par joueurs (player1_total_points_total, player2_total_points_total)
    - Côtes du Bookmarker PS (player1_oddsPS, player2_oddsPS) : afin de déterminer si cela change la donne

Preprocessing

Les variables sélectionnées pour l'entrainement du modèle sont celles évoquées ci-haut. Certaines ont été recalculées :

- Changement de type : parsing des variables match_date, player1_birthdate, player2_birthdate en datetime

- Création de nouvelles variables : année et mois de la date du match afin d'intégrer la temporalité avec une moyenne mobile

Après avoir stocké toutes les features dans un nouveau dataframe nommé X et la variable cible dans une variable y, nous avons procédé à la création d'un ensemble d'entraînement et d'un ensemble de tests à l'aide de la fonction train_test_split de sklearn dont nous avons paramétré de tel sorte que l'ensemble de test concerne 20% du dataset de base

Enfin, les méthodes MinMaxScaler et StandardScaler sont utilisées pour standardiser les variables afin de limiter l'influence des variables ayant une grande échelle ou des valeurs extrêmes.

Identification du type de problème (discrimination, clustering…) et choix de l'algorithme

Nous sommes en présence d'un problème de classification binaire. La tâche de classification consiste à prédire la classe à laquelle appartient un joueur pour un match donné.

On privilégie l'algorithme à la précision la plus élevée pour favoriser la rentabilité.

La recherche de la performance nous encourage non seulement à réaliser les classiques régression logistique, SVM linéaires ou arbre de décision, mais également les modèles neuronaux plus poussés dont les graphes.

Pour comparer les différents modèles, nous avons utilisé le taux de bonnes prédictions ('accuracy') d'un modèle car il s'agit de la métrique souvent utilisée en classification, et correspondant au nombre d'éléments bien classés sur le nombre total d'individus.

Nous avons aussi utilisé la métrique l'AUC pour évaluer la performance du modèle retenu car cette métrique est plus efficace que l'accuracy. L'aire sous la courbe (AUC : Area Under the Curve) est très utile. En un seul nombre elle résume la capacité du modèle à distinguer la classe négative de la classe positives (Gagnant / Perdant).

Cependant, à précision égale, on privilégiera le modèle le plus simple pour des raisons d'interprétabilité.

III- Modèles

1- Modèles évalués

Etant donné que nous sommes présence d'un problème de classification binaires, nous avons essayé les algorithmes suivants :

Pour modéliser notre problème de classification binaire supervisée, une technique d'optimisation de paramètres de type GridSearch sur les algorithmes de régression logistique, Random Forest et KNN afin de déterminer les meilleurs paramètres par modèle

Nous avons donc créé une grille de recherche et entrainé les différents modèles avec les paramètres optimaux obtenus

Régression logistique

La Régression linéaire nous permet d'étudier les relations entre l'ensemble de variables explicatives et une variable cible. La fonction de lien utilisée est la fonction logistique qui donne une probabilité allant de 0 à 1.

Dans le cas d'une classification, les algorithmes d'optimisation utilisés sont L_BFGS (Limite_memory BFGS), SAG (Stochastic Average Gradient), SAGA, Newton_cg (Newton Conjugate Gradient).

SVM

Le principe des Support Vector Machine (SVM) est le même pour la classification à l'aide de la régression logistique. Dans notre cas, cette méthode nous permet de prédire la classe des gagnants/perdants en se basant sur le signe de la fonction de décision $h_0(X)$ lors de la classification d'un nouvel élément.

Notre modèle prend en paramètres :

- Le kernel : fonctions noyaux à utiliser (linear, polynomial, rbf et sigmoid)
- C : paramètre de régularisation, qui doit être une constante strictement positive
- Gamma : coefficient kernel pour les fonctions noyaux.

KNN

La méthode des K-plus proches voisins (KNN) repose sur l'attribution d'une nouvelle donnée à la classe majoritaire parmi les K plus proches voisins au sens d'une distance choisie.

Pour entrainer le modèle, nous utilisons la fonction « KNeighborsClassifier » du package « neighbor » permettant de créer un classifieur KNN en lui donnant en arguments le nombre de nombre de 'voisins' Neighbors choisi entre 1 et 40.

Au passage, l'utilisation de GreadSerchcv nous aide à déterminer le nombre de voisins optimaux ainsi que la distance retenue

Arbre de décision

L'arbre de décision permet une représentation visuelle d'un algorithme de classification de données suivant différents critères qu'on appelle décisions (ou nœuds). Chaque nœud correspond à un test sur une variable d'apprentissage, et chacune des branches suivantes représente un résultat de ce test. Chaque feuille de l'arbre (nœuds terminaux) contient une valeur de la variable cible (un label dans le cas d'une classification).

Lors de l'entraînement du modèle, les nœuds sont créés à partir de tests "optimaux" par rapport à l'ensemble d'entraînement, et ce dernier prend fin lorsque les feuilles de l'arbres sont homogènes ou vérifient un certain critère d'arrêt.

Nous avons choisi les paramètres suivants :

- criterion ='entropy' : critère d'arrêt
- max_depth=4 : pour spécifier le nombre maximum de points de séparation possible avant d'atteindre un nœud "feuille
- random_state=123 : pour la reproductibilité des résultats

Random Forest

La méthode Random Forest est un cas particulier du Bagging appliqué aux arbres de décision (CART). En effet, de nombreux petits arbres de classification sont produits sur une fraction aléatoire de données, ensuite l'algorithme fait ensuite voter ces arbres de classification peu corrélés afin de déduire l'ordre et l'importance des variables explicatives.

Les avantages des modèles Random Forest sont :

- L'efficacité sur les données de grandes dimensions
- Leur robustesse au niveau de l'identification des valeurs aberrantes

Par ailleurs, de manière générale, elles évitent l'overfitting

Pour entrainer notre modèle, nous utilisons le classifieur « RandomForestClassifier » avec les paramètres suivants :

- Le nombre d'arbres de la forêt aléatoire : `n_estimators`
- Le nombre minimum d'échantillons requis pour se trouver au niveau d'un nœud feuille : `min_samples_leaf`
- Le nombre de features à prendre en compte lors de la recherche du meilleur split : `max_features`

Gradient Boosting

Dans la continuité des decision tree et des random forest, nous avons également testé le gradient boosting. Le Gradient Boosting Tree est un autre algorithme de boosting dont l'objectif est d'entraîner une forêt d'arbre de manière séquentielle afin que chaque nouvel

arbre compense les faiblesses de la forêt. Cette fois l'algorithme se concentre sur les résidus des précédents arbres plutôt que sur les poids des exemples.

Les résidus sont la différence entre la prédiction d'un modèle et la valeur cible. Si l'on ajoute le résidu à la valeur prédite on atteint la valeur cible. L'idée ici est de prédire les résidus afin de les ajouter à la forêt pour avoir une prédiction plus précise de la variable cible. La prédiction d'une forêt sera l'addition de la prédiction de chacun des estimateurs.

Dense neural network

Le modèle choisi comporte plusieurs couches :

- Une couche d'entrée constitués de 80 neurones qui reçoivent les valeurs d'entrée et les transmettent aux neurones cachés
- Deux couches cachées : une deuxième couche avec 50 neurones et une troisième couche avec20neurones.
- Une couche de sortie : avec 1 neurones car nous souhaitons prédire 2 classes

Etant donné que nous sommes en face d'un problème de classification binaire, nous avons choisi une fonction d'activation « softmax ».

```
Model: "model_7"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 Input (InputLayer)          [(None, 20)]              0

 Dense_1 (Dense)             (None, 80)                1680

 Dense_2 (Dense)             (None, 50)                4050

 Dense_3 (Dense)             (None, 20)                1020

 Dense_4 (Dense)             (None, 1)                 21

=================================================================
Total params: 6,771
Trainable params: 6,771
Non-trainable params: 0
_________________________________________________________________
```

Une fois le modèle défini, nous le compilons avec une fonction de perte binary_crossentropy adapté aux problèmes de classification binaire.

Nous entrainerons le modèle avec 26 epochs composés de 32 batchs

La fonction fit () retourne deux valeurs :

- La perte du modèle sur l'ensemble de données
- La précision du modèle sur l'ensemble de données

Idéalement, la perte devrait être nulle et la précision égale à 1

Graph neural network

Le knn est notre dernier modèle évalué dans l'objectif d'obtenir une meilleure précision de modèle.

L'hypothèse posée est que les modèles précédents ne sont pas en mesure de tirer parti des relations entre joueurs : Si joueur A bat B qui bat C, on peut raisonnablement supposer que A est avantagé contre C.

L'objectif est d'utiliser les graphes pour faire apparaitre ces relations. Les joueurs sont des nœuds, les matchs sont des edges, et l'objectif est de prédire la direction de l'edge qui représente la victoire d'un des 2 joueurs.

Notre graphe est donc directed avec des features sur les nodes et sur les edges, les layers GATConv seront utilisées pour tirer parti des edge_attr.

2 – Comparaison des modèles

Le tableau ci-dessous affiche les différentes performances des modèle entrainés :

Modèle	Accuracy	ROI
Logistic regression	Training Accuracy: 71.85 Test Accuracy: 71.35	money spent: 7395 € money won: -1648.56 € TOTAL: -6845.60 €
Logistic regression with moving average	Training Accuracy: 62.97 Test Accuracy: 61.37	money spent: 5164 € money won: -1532.91 € TOTAL: -6696.91 €
KNN	Training Accuracy: 71.31 Test Accuracy: 67.58	money spent: 7395 € money won: -2244.25 € TOTAL: -9639.25 €
Decision Tree	Training Accuracy: 69.50 Test Accuracy: 69.40	money spent: 7395 € money won: -2455.06 € TOTAL: -9850.06 €
Random Forest	Training Accuracy: 97.82 Test Accuracy: 66.86	money spent: 7395 € money won: -2394.61 € TOTAL: -9789.61 €
Gradient Boosting	Training Accuracy: 69.02 Test Accuracy: 68.97	
SVM	Training Accuracy: 73.60 Test Accuracy: 67.90	money spent: 7395 € money won: -2440.25 € TOTAL: -9835.25 €
SVM with moving average	Training Accuracy: 81.09 Test Accuracy: 57.82	money spent: 5164€ money won: -1532.91 € TOTAL: -6696.91 €
Dense neural network	Training Accuracy: 68.89 Test Accuracy: 68.11	money spent: 7395 € money won: -2443.51 € TOTAL: -9838.51 €
Graph neural network	Training Accuracy: 57 Test Accuracy: 57	

Observations :

- ⇨ La plupart des modèles plafonnent à 70% d'accuracy, ce qui s'avèrera insuffisant pour obtenir un ROI positif
- ⇨ L'ajout de nouvelles features via le web scrapping apporte des performances marginales au modèle, les features les plus pertinentes sont déjà présentes.
- ⇨ De manière assez paradoxale, il a été observé que des modèles avec précisions moindre obtenaient un ROI plus élevé. On peut supposer que l'alignement du modèle sur la cote des bookmakers provoquent un style de jeu très prudent qui implique de très faibles gains en cas de victoire et des pertes élevées en cas de défaites.
- ⇨ A performance égale, il est plus judicieux de retenir l'un des modèles les plus simple à expliquer, comme la régression logistique par exemple.

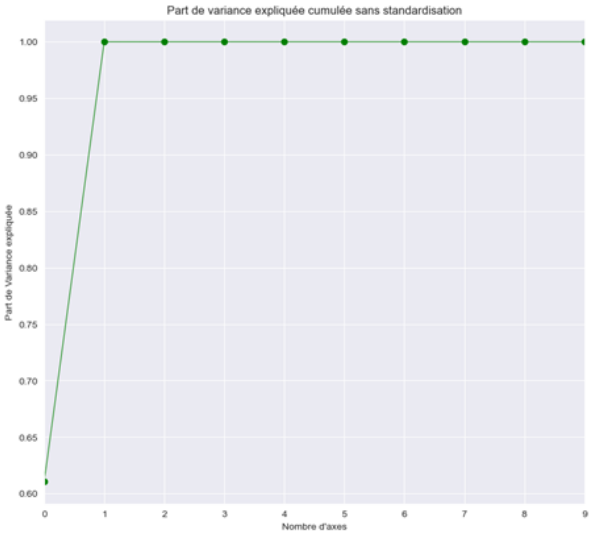
3 – Réduction de dimension

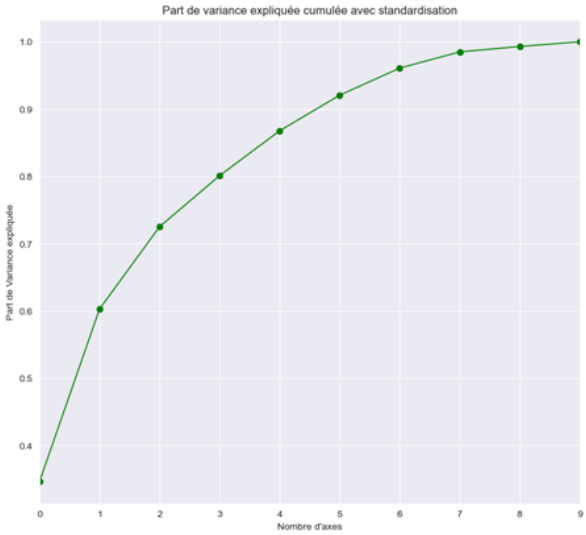
La réduction de dimension n'a pas pour but d'améliorer l'accuracy mais d'améliorer les performances d'exécution du modèle qui aurait trop de paramètres. Nous ne sommes pas dans cette situation, néanmoins la réduction de dimension a été implémentée pour permettre la mise en pratique des connaissances du cours.

Modèle	**Accuracy**
PCA	Training Accuracy PCA: 62.00
t-SNE	Training Accuracy TSNE: 60.12
PCA avec selector Kbest	Training Accuracy PCA - Kbest: 67.26
PCA avec selector Percentile	Training Accuracy PCA - Percentile: 64.73

Le sélecteur Kbest est le sélecteur ayant la meilleure efficacité lorsque l'on effectue un ACP. De plus nous pouvons noter que sa présence n'impacte pas l'efficacité des modèles mais peut légèrement améliorer l'efficacité du modèle dans certains cas (accuracy KNN sans sélecteur : 0.65 et avec sélecteur 0.68 Régression Logistique sans sélecteur : 0.6921 et 0.69196 avec le sélecteur KBest). Autrement dit notre sélecteur effectue sa tache correctement.

Au cours de l'analyse en composante principale (ACP), nous notons que la répartition de l'information de notre dataset à travers nos axes est complètement différente avec et sans la normalisation. Comme le présente les graphiques suivants :

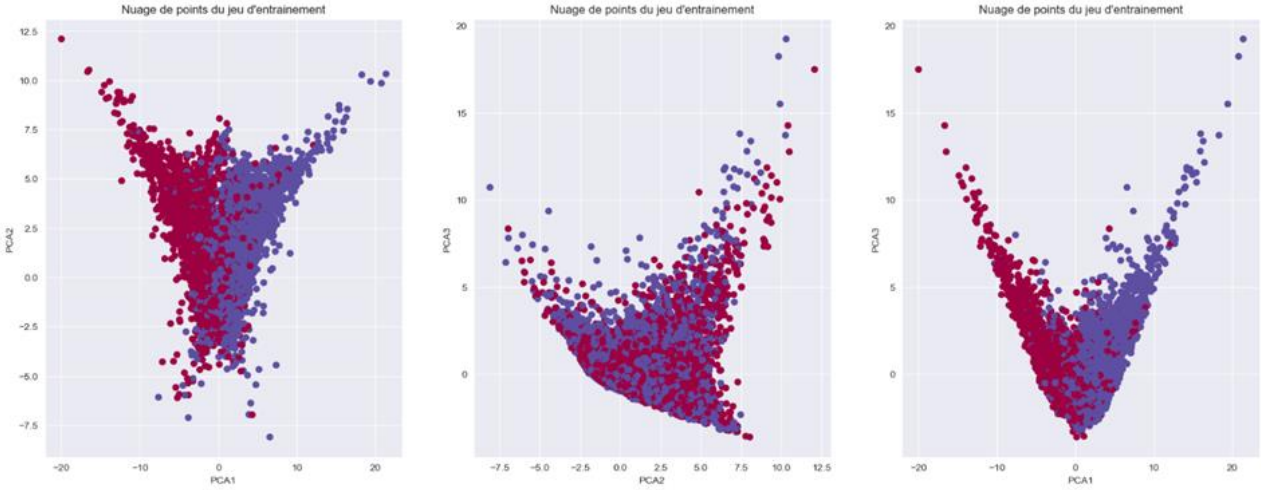




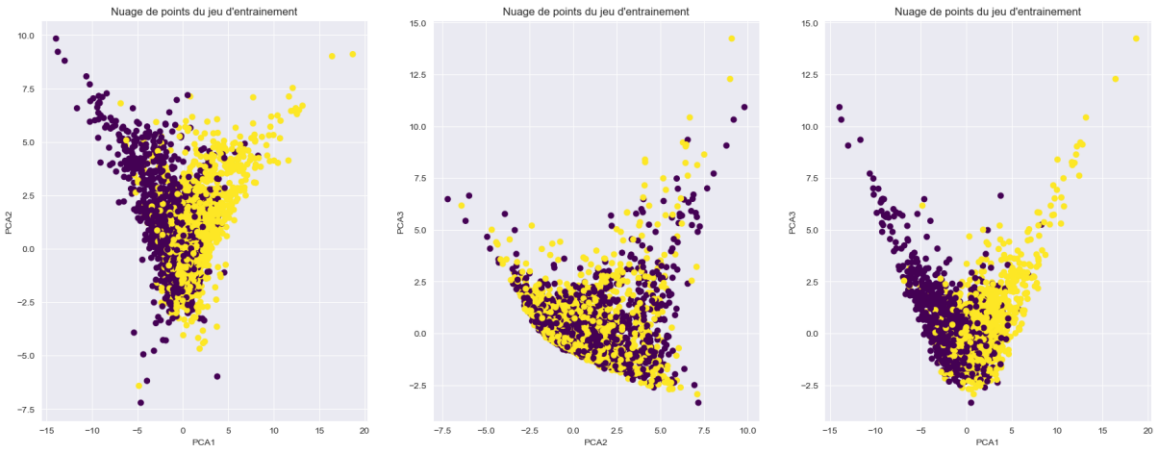
Notre jeu de donnée présentant des variables à des échelles différentes et une taille assez conséquente ne peut pas réussir à capter l'intégralité de l'information avec seulement deux axes. Nous allons donc nous référer au deuxième graphique afin d'effectuer notre ACP.

Au cours de notre ACP, nous pouvons voir que les premiers axes nous permettent de répartir nos nuages de points issus du jeu d'entrainement et de test en fonction des victoires ou des défaites des joueurs représenté en couleur.

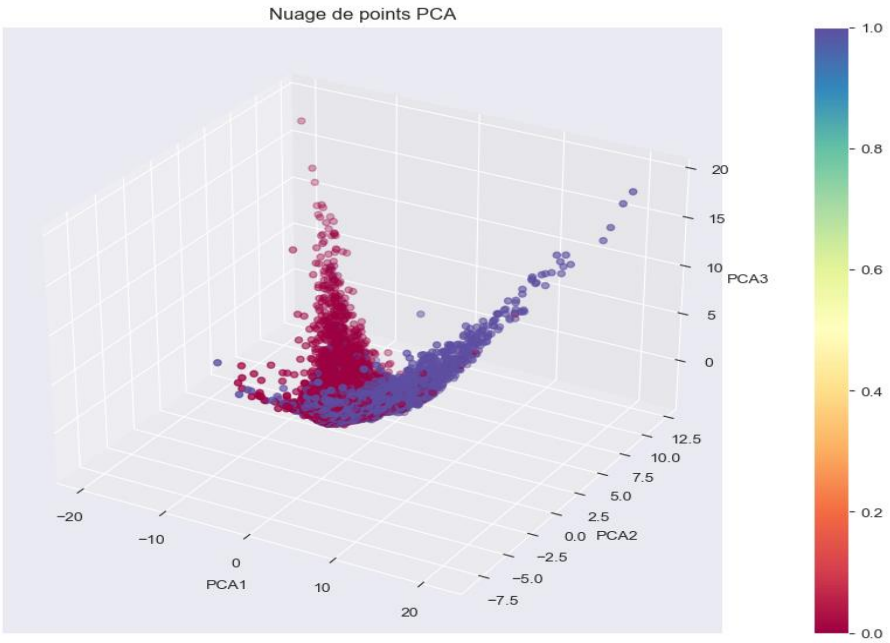
Nuage de points du jeu d'entrainement représenté sur les 3 premiers axes entre elles :



Nuage de points du jeu de test :



Représentation globale de notre nuage de points :



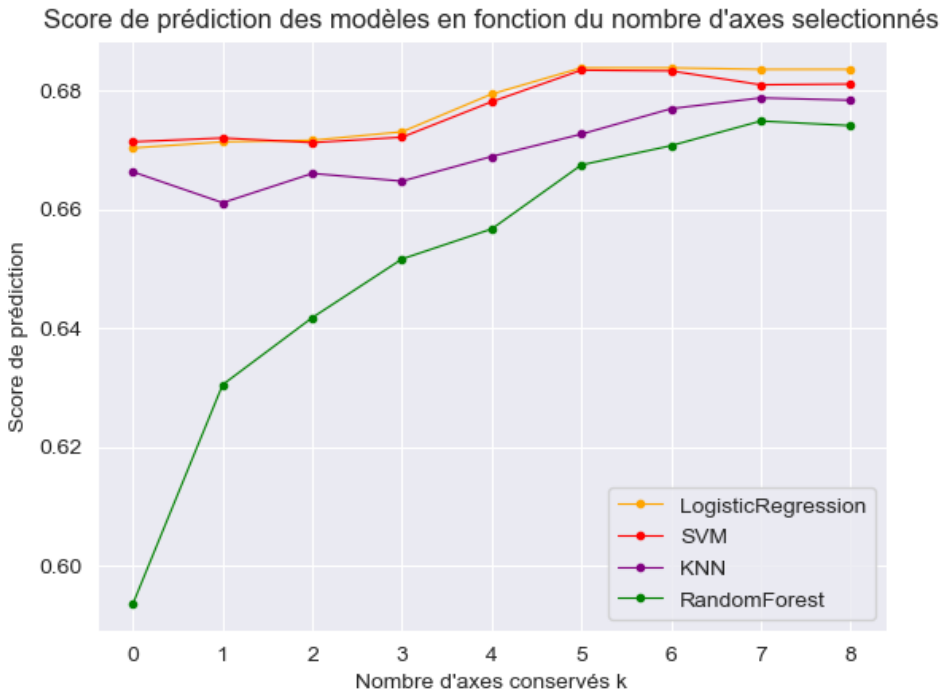
La représentation de l'ACP est très bonne cependant la caractérisation de la nature des axes permettant de séparer nos joueurs est délicate et nous limite dans l'approfondissement de notre analyse. Cela nous aurait permis de déterminer quel facteur primordiale détermine la victoire ou défaite d'un joueur représenté en couleur.

Après avoir effectué un GridSearch afin d'obtenir les coefficients optimaux de chaque modèle sur ce jeu de donnée, nous avons pu nous rendre compte que la normalisation de notre jeu de donnée n'a pas impacté l'efficacité de nos modèles optimaux. Ces scores sont pris avant la PCA pour voir l'effet de la normalisation :

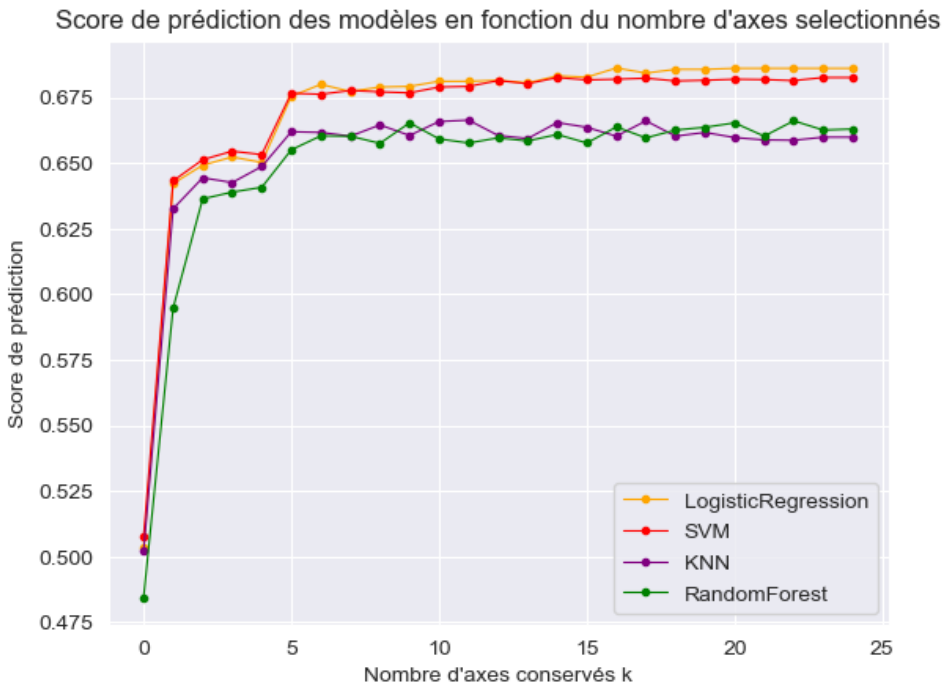
Modèle	**Accuracy**
Regression logistique	Score sur donnée réduite:0.678 Score sur donnée complète:0.686
KNN	Score sur donnée réduite: 0.669 Score sur donnée complète:0.671
RandomForest	Score sur donnée réduite:0.659 Score sur donnée complète:0.667
SVM	Score sur donnée réduite : 0.65 Score sur donnée complète : 0.683

Score de prédiction de performance du modèle avec le sélecteur KBest et la PCA :

Si on analyse le graphique si dessous nous pouvons voir que la Régression Logistique est le modèle le plus performant avec le SVM. Cependant nous pouvons noter que nos modèles n'arrivent pas à dépasser le plafond d'efficacité de 70% et stagne quel que soit le nombre d'axes qu'on leur rajoute. La Régression Logistique conserve 6 axes avec un meilleur score de 0.684, le SVM conserve 6 axes avec un meilleur score de 0.683, le KNN garde 8 axes, avec un score de 0.679 et 9 axes avec un score de 0.67 pour le RandomForest.



Si nous revenons sur le choix de sélecteur et que nous prenons le sélecteur Percentile. Alors nous avons des scores de prédictions suivants pour nos modèles :



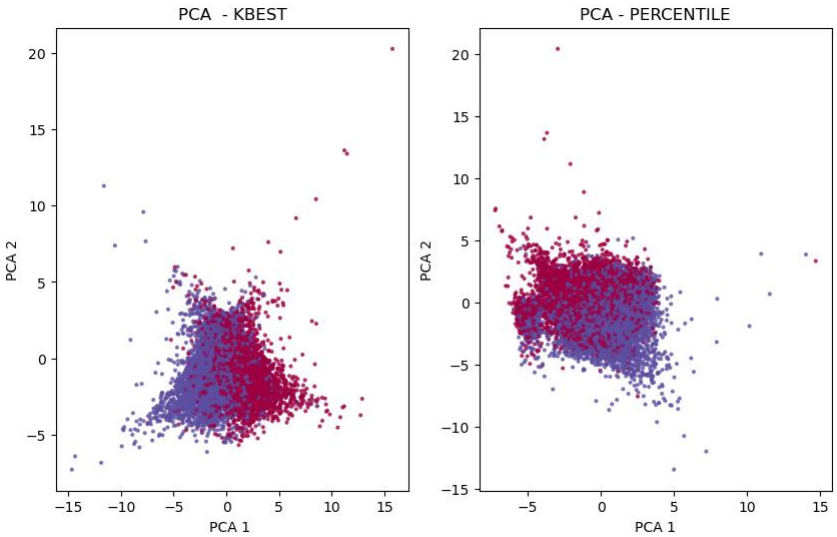
Dans ce cas-ci le sélecteur Percentile conserve plus de variable. Il est donc normal que nous ayons donc plus d'axes sur le graphique. La tendance reste conservée car nous avons le modèle Logistique en tête. La Régression Logistique conserve 17 axes avec un score de 0.686, le modèle SVM conserve 15 axes et détient un score de 0.68. Pour le KNN, le nombre d'axe est

de 12 avec une efficacité de 0.67 et le RandomForest garde 23 axes avec la même efficacité : 0.67.

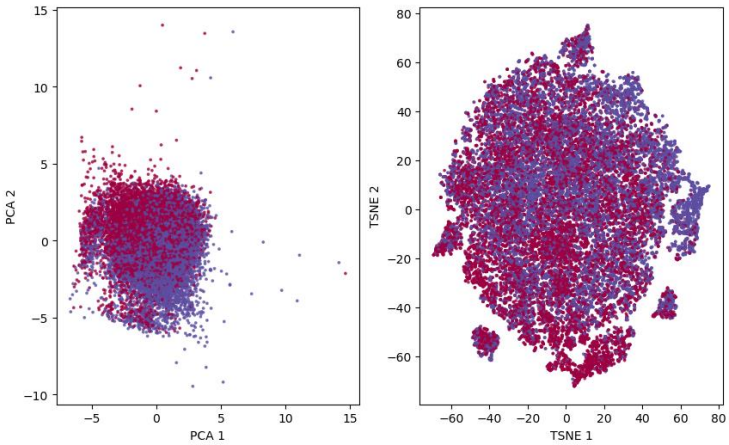
Ces résultats peuvent s'expliquer par une augmentation du nombre de variables choisies par le sélecteur. En effet cela impactera donc le choix des axes pris en compte par l'ACP ainsi donc que l'efficacité final de notre modèle.

Résultats :

- ⇨ La PCA couplé à au sélecteur SelectKBest, nous permet de distinguer deux classes à savoir le groupe des gagnants et des perdants. De plus, la performance du modèle est améliorée.



- ⇨ L'algorithme t-SNE permet d'identifier la présence de structure mais reste moins performant que la PCA



4 – Interprétation du modèle (Logistic regression)

Le modèle que nous avons choisi de retenir pour ce chapitre est la régression logistique.

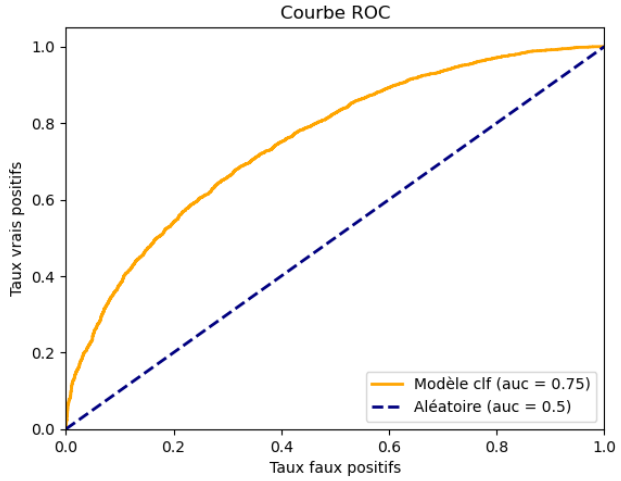
```
LogisticRegression ( max_iter=2000, random_state=22, C=0.01778279410038923,
solver=" lbfgs")
```

Matrice de confusion

```
Classe prédite     0     1
Classe réelle
0               1742  1993
1                536  3124
```

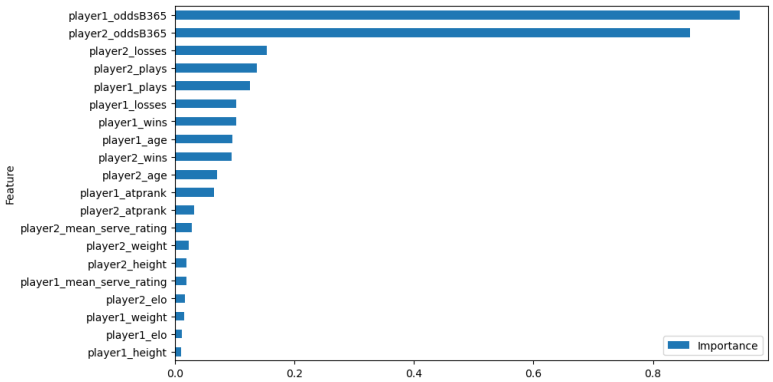
La matrice de confusion nous permet d'observer que notre modèle ne prédit pas correctement les perdants.

Précision du modèle



L'AUC ROC de notre modèle se situe bien au-dessus de celui d'un modèle non-informatif et en-dessous de celui d'un modèle parfait. Avec 75% d'AUC ROC, on en déduit que notre modèle est assez performant.

Importance de variable obtenue avec la Régression logistique



	Feature	Importance
9	player1_oddsB365	0.945039
19	player2_oddsB365	0.861842
14	player2_losses	0.154747
12	player2_plays	0.137298
2	player1_plays	0.125669
4	player1_losses	0.102976
3	player1_wins	0.102713
0	player1_age	0.096706

L'analyse de l'importance des features obtenus par le modèle de la Régression logistique, il en découle que les côtes du Bookmaker contribuent en grande dans la prédiction du vainqueur du tournoi.

Significativité des variables

Objectif : Prédire le vainqueur du match de tennis

H0 : Il n'y a pas d'effet entre une variable explicative donnée et la variable cible

H1 : Il a un effet entre une variable explicative donnée et la variable cible

Pour tester la significativité du modèle, nous avons utilisé le module statmodels avec la fonction Logit.

```
                           Logit Regression Results
==============================================================================
Dep. Variable:         winner_player1   No. Observations:                29577
Model:                          Logit   Df Residuals:                    29558
Method:                           MLE   Df Model:                           18
Date:                Thu, 28 Sep 2023   Pseudo R-squ.:                  0.1596
Time:                        14:50:47   Log-Likelihood:                -17228.
converged:                       True   LL-Null:                       -20501.
Covariance Type:            nonrobust   LLR p-value:                     0.000
==========================================================================================
                             coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------------------
player1_age               -0.1046      0.020     -5.328      0.000      -0.143      -0.066
player1_atprank           -0.0579      0.019     -3.115      0.002      -0.094      -0.021
player1_plays             -0.3960   2.88e+06  -1.37e-07      1.000   -5.65e+06    5.65e+06
player1_wins               0.7031   2.49e+06   2.83e-07      1.000   -4.87e+06    4.87e+06
player1_losses             0.2734   1.04e+06   2.63e-07      1.000   -2.03e+06    2.03e+06
player1_elo               -0.0283      0.027     -1.047      0.295      -0.081       0.025
player1_mean_serve_rating  0.0162      0.018      0.903      0.367      -0.019       0.051
player1_height             0.0087      0.021      0.422      0.673      -0.032       0.049
player1_weight            -0.0150      0.021     -0.703      0.482      -0.057       0.027
player1_oddsB365          -1.0069      0.043    -23.376      0.000      -1.091      -0.922
player2_age                0.0803      0.020      4.070      0.000       0.042       0.119
player2_atprank            0.0236      0.016      1.513      0.130      -0.007       0.054
player2_plays             -0.7557   2.89e+06  -2.62e-07      1.000   -5.66e+06    5.66e+06
player2_wins               0.3970   2.49e+06   1.59e-07      1.000   -4.88e+06    4.88e+06
player2_losses             0.0021   1.04e+06   2.05e-09      1.000   -2.03e+06    2.03e+06
player2_elo                0.0173      0.027      0.630      0.529      -0.037       0.071
player2_mean_serve_rating -0.0260      0.018     -1.426      0.154      -0.062       0.010
player2_height             0.0229      0.021      1.084      0.278      -0.019       0.064
player2_weight            -0.0255      0.022     -1.165      0.244      -0.068       0.017
player2_oddsB365           0.9102      0.043     21.315      0.000       0.826       0.994
==========================================================================================
```

Uniquement 5 variables (player1_age, player1_atprank, player1_oddsB365, player2_age et player2_oddsB365) sont significatives.

Nous avons donc utilisé une méthode de sélection de variables afin d'essayer de trouver un modèle qui aurait plus de features. Pour ce faire, nous avons utilisé la méthode de sélection de variables backward en indiquant au sélecteur que nous souhaitons garder 13 variables.

```
from sklearn.feature_selection import SequentialFeatureSelector
sfl= SequentialFeatureSelector(rl_opt,  direction='backward', n_features_to_select=13, n_jobs=-1)
sfl.fit(X_train_scaler, y_train)
```

Résultat :

Features selected by forward sequential selection: ['player1_age', 'player1_atprank', 'player1_elo', 'player1_weight', 'player1_oddsB365', 'player2_age', 'player2_atprank', 'player2_plays', 'player2_wins', 'player2_losses', 'player2_height', 'player2_weight', 'player2_oddsB365']

Nous avons recalculé la significativité des variables de ce deuxième modèle.

Modèle 2 – Issue de la sélection des variables

```
                           Logit Regression Results
==============================================================================
Dep. Variable:         winner_player1   No. Observations:                29577
Model:                          Logit   Df Residuals:                    29564
Method:                           MLE   Df Model:                           12
Date:                Thu, 28 Sep 2023   Pseudo R-squ.:                  0.1593
Time:                        16:30:47   Log-Likelihood:                -17235.
converged:                       True   LL-Null:                       -20501.
Covariance Type:            nonrobust   LLR p-value:                     0.000
=====================================================================================
                        coef    std err          z      P>|z|      [0.025      0.975]
-------------------------------------------------------------------------------------
player1_age          -0.0631      0.013     -4.853      0.000      -0.089      -0.038
player1_atprank      -0.0644      0.018     -3.499      0.000      -0.101      -0.028
player1_elo           0.0028      0.021      0.136      0.892      -0.037       0.043
player1_weight        0.0014      0.013      0.105      0.917      -0.024       0.027
player1_oddsB365     -1.0152      0.040    -25.101      0.000      -1.094      -0.936
player2_age           0.0724      0.019      3.732      0.000       0.034       0.110
player2_atprank       0.0193      0.015      1.261      0.207      -0.011       0.049
player2_plays       -15.0221      6.944     -2.163      0.031     -28.633      -1.411
player2_wins         12.8334      5.965      2.151      0.031       1.142      24.525
player2_losses        5.1043      2.415      2.114      0.035       0.372       9.837
player2_height        0.0188      0.021      0.909      0.363      -0.022       0.059
player2_weight       -0.0375      0.020     -1.837      0.066      -0.077       0.003
player2_oddsB365      0.9215      0.042     22.066      0.000       0.840       1.003
=====================================================================================
```

Les variables player1_elo, player1_weight, player2_atprank, player2_weight, playe2_height n'étant pas significatives, nous allons les enlever du modèle manuellement et retester un nouveau modèle avec le reste des variables.

Modèle 3

```
                           Logit Regression Results
==============================================================================
Dep. Variable:         winner_player1   No. Observations:                29577
Model:                          Logit   Df Residuals:                    29569
Method:                           MLE   Df Model:                            7
Date:                Thu, 28 Sep 2023   Pseudo R-squ.:                  0.1592
Time:                        16:36:05   Log-Likelihood:                -17237.
converged:                       True   LL-Null:                       -20501.
Covariance Type:            nonrobust   LLR p-value:                     0.000
=====================================================================================
                        coef    std err          z      P>|z|      [0.025      0.975]
-------------------------------------------------------------------------------------
player1_age          -0.0627      0.013     -4.825      0.000      -0.088      -0.037
player1_atprank      -0.0613      0.018     -3.472      0.001      -0.096      -0.027
player1_oddsB365     -1.0235      0.040    -25.700      0.000      -1.102      -0.945
player2_age           0.0753      0.018      4.079      0.000       0.039       0.111
player2_plays       -16.4763      6.714     -2.454      0.014     -29.635      -3.318
player2_wins         14.0400      5.780      2.429      0.015       2.712      25.368
player2_losses        5.6234      2.330      2.413      0.016       1.056      10.191
player2_oddsB365      0.9297      0.038     24.715      0.000       0.856       1.003
=====================================================================================
```

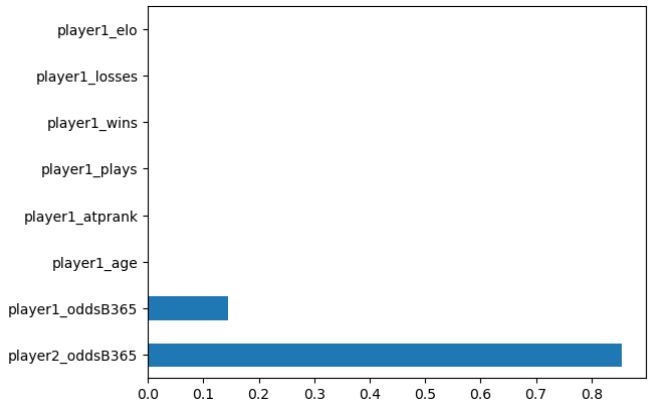
Pour notre projet, nous retiendrons donc ce modèle.

Performance du modèle

Modèle	Accuracy	Gain
Régression logistique après suppression de variable non significatives	Training Accuracy: 69.12 Test Accuracy: 67.61	We bet 1€ on all test matches money spent: 7395 € money won: -2441.16 € TOTAL: -9836.16 €
Régression logistique avec le dataset de base	Training Accuracy: 69.17 Test Accuracy: 67.94	We bet 1€ on all test matches money spent: 7395 € money won: -2444.96 € TOTAL: -9839.96 €

La performance du modèle après sélection de variables non significative ainsi que le gain obtenu sont légèrement inférieures à ceux du modèle avec l'ensemble des variables du dataset de base.

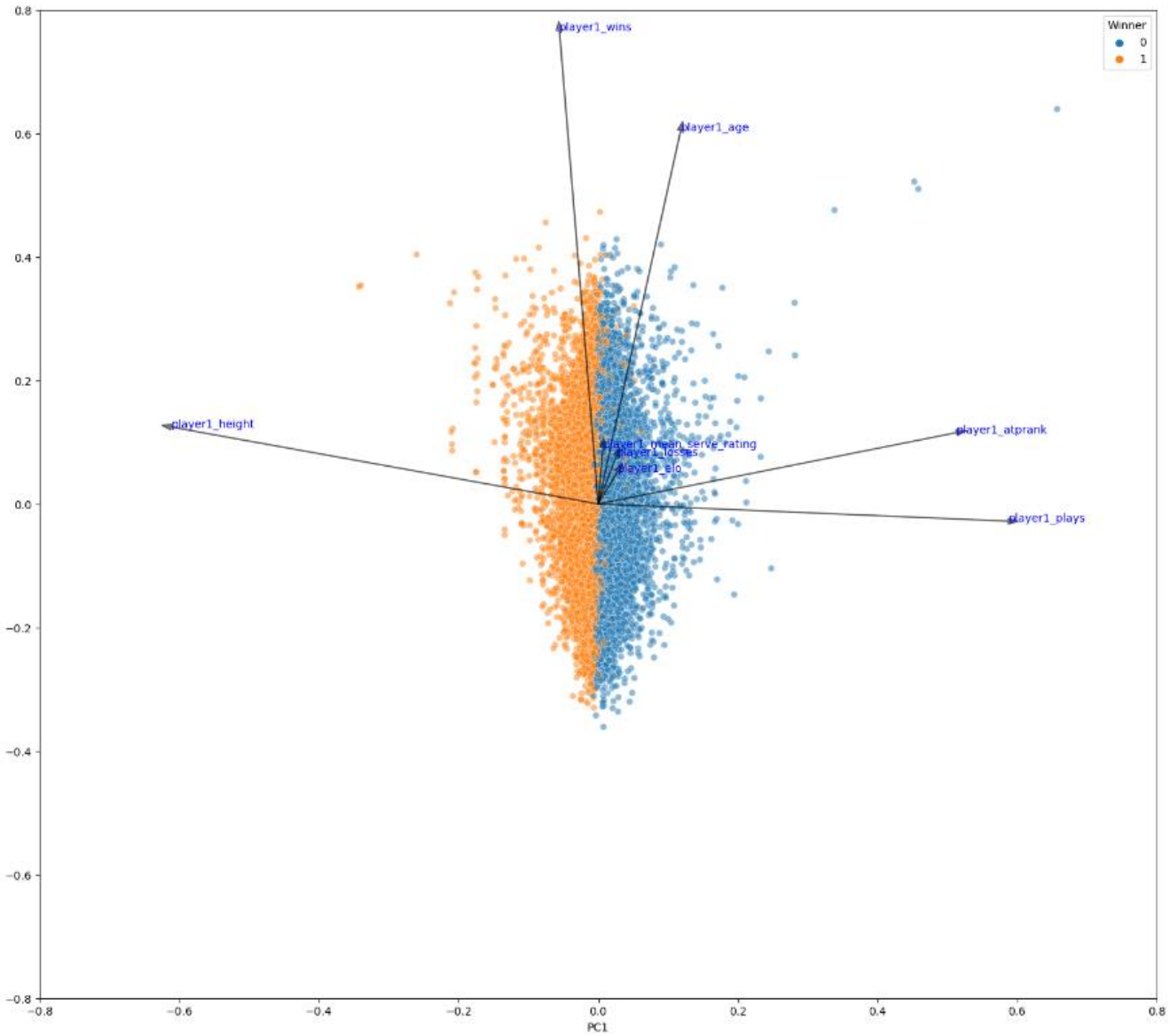
Interprétabilité du modèle à l'aide de l'arbre de décision (avec le modèle final après suppression des variables non significatives)



	Importance
player2_oddsB365	0.855607
player1_oddsB365	0.144393
player2_atprank	0.000000
player2_weight	0.000000
player2_height	0.000000
player2_mean_serve_rating	0.000000
player2_elo	0.000000
player2_losses	0.000000

L'arbre de décision suggère que les côtes du Bookmaker suffirait pour prédire le vainqueur du tournoi.

Interprétabilité du modèle à l'aide du PCA (avec le modèle final après suppression des variables non significatives)

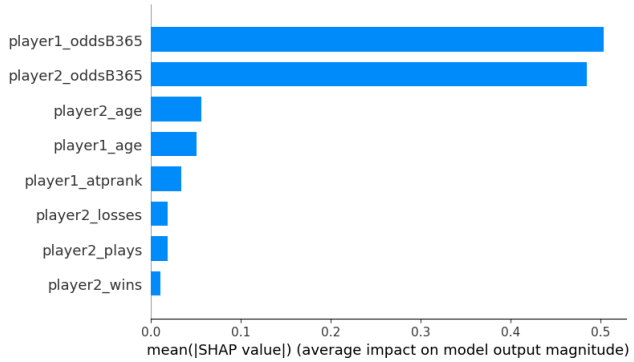


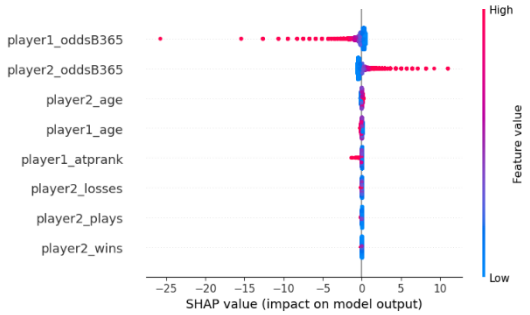
La modélisation du modèle ”Régression logistique” avec réduction de dimension par PCA nous permet de distinguer les 2 classes à prédire grâce à la 2ème composante principale (PCA2).

Interprétabilité de type SHAP

SHAP (SHapley Additive exPlanations) est un excellent package proposant une approche unifiée pour expliquer le résultat de tout modèle de Machine Learning. SHAP relie la théorie des jeux aux explications locales, en se basant sur les valeurs de Shapley. De manière générale, les prédictions des modèles peuvent être expliquées en supposant que chaque variable explicative est un "joueur" dans un jeu où la prédiction est le paiement. La valeur de Shapley- une méthode issue de la théorie des jeux coopératifs- nous indique comment répartir équitablement le "gain" entre les variables.

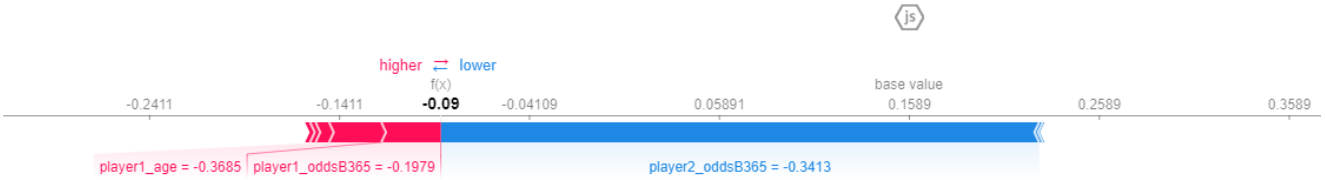
Importance obtenue avec SHARP et Densité des valeurs sharp





Les cotes, l'âge et le rang ATP des joueurs ont plus de poids lors de la prédiction du vainqueur du match (voir le graphique de densité à droite)

Interprétation pour quelques individus



Conclusion

L’utilisation des features fournies dans le dataset initial nous permet très vite d’obtenir une accuracy de l’ordre de 70% avec des modèles simples. En revanche, malgré cette précision, le retour sur investissement reste négatif.

L'interprétabilité des modèles réalisés nous permet d’identifier que la cote des joueurs est un des plus grands facteurs d’influence pour notre classification binaire : nous sommes alors alignés sur la prédiction du bookmaker, qui nous fait gagner certes 70% du temps, mais nous fait perdre davantage en 30% de défaites.

Sans cette cote, nos modèles descendent à 65% de précision et l’on observe que cette précision n’est pas suffisante pour garantir un retour sur investissement.

A ce stade, plusieurs stratégies peuvent êtres implémentés : faut-il parier contre le bookmaker : fort gain a faible fréquence ; ou parier avec lui : faible gain à forte fréquence.

En appliquant le modèle sur certains matchs uniquement (confidence du modèle, nombres de matchs minimums pour les joueurs...), plutôt que sur l’ensemble, on parvient de temps en temps à un retour positif, mais le nombre de parties jouées est alors marginal, tout comme le gain, et nos critères de sélections ne résistent pas aux validations croisées.

In fine il revient d’abord de connaitre le gagnant du match, et si la cote nous permet de gagner 5%, mieux vaut ne pas l’exclure. Le reste de notre temps sera consacré à un travail de webscrapping pour ajouter de nombreuses features aux modèles, ainsi que l’étude de nombreux autres modèles.

Dans notre dernier modèle le plus abouti, nous noterons notamment que l'âge des joueurs est le facteur de 3ème importance, alors que celui-ci n’était pas présent dans notre jeu de données initial.

Enfin, certaines pistes comme le graph neural network semble très prometteuses car elles permettent une meilleure représentation des relations entre joueurs dont les autres modèles sont incapables. Sujet complexe qui aurait mérité un mois supplémentaire pour en voir son aboutissement.

De manière générale, le projet pari qui nous a été proposé nous a permis une approche progressive du Machine Learning, puis une montée en complexité au fur et à mesure des nouveaux modèles étudiés. Nous avons pu mettre en pratique tous les éléments du cours et nous en affranchir avec nos propres analyses et modèles.